API Documentation

> Source of truth: the NestJS controllers under `apps/api/src/modules/**`. Every endpoint below was read in code at commit `8de48cf`. Interactive Swagger is served by the running API at **`/api/docs`** (`apps/api/src/main.ts`).
>
> Legend — **Auth**: `Public` = `@Public()` (no token); `Any` = any authenticated role; otherwise the roles from `@Roles(...)`. Role ids: 1 Candidate (Subscriber), 2 QC1, 3 QC2, 4 Client (Employer), 5 Admin, 6 Subscription, 7 Q3.

Contents

1. Cross-cutting behaviour
2. Auth
3. Candidates
4. Files
5. Jobs (public search + apply)
6. Employers / Clients
7. Recruitment (legacy pipeline + shared hiring actions)
8. Q1 screening
9. Q2 matching
10. Q3 hiring operations
11. Admin
12. Payments
13. Exports
14. Notifications
15. Companies, site documents, health
16. Background jobs
17. External integrations
18. Frontend calls with no backend route

---

1. Cross-cutting behaviour

Concern	Implementation	File
Global prefix	`/api`	`apps/api/src/main.ts`
Authentication	`JwtAuthGuard` is a global `APP_GUARD`. Every route needs `Authorization: Bearer <access token>` unless it is `@Public()`. The token must carry `type: 'access'`.	`apps/api/src/common/guards/jwt-auth.guard.ts`
Authorization	`RolesGuard` (global). A method-level `@Roles` **replaces** the class-level one (`getAllAndOverride`). A route with no `@Roles` admits any authenticated role.	`apps/api/src/common/guards/roles.guard.ts`
Rate limit	`ThrottlerGuard` (global), `THROTTLE_LIMIT_PER_MIN` per client IP (default 60). Tighter `@Throttle` overrides on auth, uploads and payments. `trust proxy = 1` so the IP is nginx's forwarded address.	`apps/api/src/app.module.ts`, `main.ts`
Validation	Global `ValidationPipe` with `whitelist`, `forbidNonWhitelisted`, `transform`, `enableImplicitConversion`. Unknown body fields → 400.	`main.ts`
Error shape	`AllExceptionsFilter`: `{statusCode, message, error, timestamp, path, ...extra}` (extra carries e.g. `retryAfterSeconds`, `attemptsRemaining`). Non-HTTP exceptions → 500. **Prisma errors are not mapped** (FK / unique violations surface as 500).	`apps/api/src/common/filters/http-exception.filter.ts`
Security headers	`helmet()`; CORS from `CORS_ORIGIN` (comma list) with credentials.	`main.ts`
BigInt	Serialised as JSON numbers (`BigInt.prototype.toJSON`).	`main.ts`
Request log	`METHOD url status ms` for every route except `/api/health`.	`apps/api/src/common/middleware/request-logger.middleware.ts`
Raw body	`/api/payments/webhook` receives the body as raw text (for signature verification).	`main.ts`
CSRF	`GET /api/auth/csrf-token` issues an `XSRF-TOKEN` cookie and the web client sends `X-CSRF-Token`, but **no server code validates it** (see TESTING_AND_ISSUES).	`apps/api/src/modules/auth/csrf.controller.ts`

Pagination conventions differ per module: most list endpoints take `page` (≥1) and `pageSize` (≤100) and return either `{rows,total}` or `{items,total,page,pageSize,pageCount}`.

---

2. Auth — `/api/auth`

Controller `apps/api/src/modules/auth/auth.controller.ts`, service `auth.service.ts`, OTP `otp.service.ts` (Redis).

Method + path	Auth	Throttle	Request	Response	Errors
POST `/auth/login`	Public	5/min	`{userName, password}` — userName may be the login name, the mobile, or an email	`{user, accessToken, refreshToken}`	401 invalid credentials
POST `/auth/refresh`	Public	global	`{refreshToken}`	new `{user, accessToken, refreshToken}` (rotation; old refresh token is single-use)	401 invalid / revoked / user inactive
POST `/auth/logout`	Public	global	`{refreshToken?}`	204	none (bad tokens silently ignored)
POST `/auth/register`	Public	5/min	`{fullName, email, mobile, password (8–72), countryCode?}`; Indian mobiles must match `^[6-9]\d{9}$`	`{otpRequired, registrationToken, email, expiresInSeconds:600, resendAfterSeconds:60, maxAttempts:5, devCode?}` — `devCode` only outside production	409 email/mobile taken; 429 cooldown; 503 email enqueue failed
POST `/auth/verify-otp`	Public	10/min	`{registrationToken (64 hex), code (6 digits)}`	`{user (isOnboarded:false), accessToken, refreshToken}`	400 wrong / expired / too many attempts; 409 taken meanwhile
POST `/auth/resend-otp`	Public	5/min	`{registrationToken}`	same as register (minus token)	400 expired; 429; 503
POST `/auth/forgot-password`	Public	5/min	`{userName}` (username or mobile, **not email**)	always `{message}`	—
POST `/auth/reset-password`	Public	global	`{token, newPassword (min 8)}`	`{message}`; revokes all sessions	400 invalid / used / expired
POST `/auth/register-employer`	Public	3/min	`{companyName, emailCompany, emailHR?, contactNumberCompany?, contactNumberHR?, location?, aboutCompany?, industryType?, website?, companyLogo?}`	`{id, status:'Pending'}` — no login is created	409 duplicate pending/approved registration
GET `/auth/me`	Any	global	—	`AuthUser {userId, userName, fullName, email, roleId, isOnboarded}`	401
GET `/auth/csrf-token`	Public	global	—	`{csrfToken}` + cookie	—

Tokens: access and refresh tokens share one `jti` and are signed with separate secrets (`JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`; TTL `JWT_ACCESS_TTL` default 15m, `JWT_REFRESH_TTL` default 7d). `tblSecActiveSessions` is an allow-list for refresh tokens only; the access-token guard does not consult it.

DB writes: login → `tblSecActiveSessions`, `tblSecUserLogin`, `tblAuditLog`. verify-otp → one `$transaction` creating `tblSubscriberRegistration`, `tblSubscriberCVDetails`, `tblSubscriberStatusHistory` (status 1), `tblSecUser`, `tblSecMapUserRoles`. register-employer → `tblEmployerRegistration` (status `Pending`).

---

3. Candidates — `/api/candidates`

Controller `apps/api/src/modules/candidates/candidates.controller.ts`, class-level `@Roles(Subscriber)`. Service `candidates.service.ts`. Every route resolves the candidate through `tblSecUser.SubscriberID` and returns **404 "This login is not linked to a candidate profile"** when it is null (true for migrated legacy candidates).

All rows are scoped by `subscriberID` (updates use `updateMany` with the owner filter, so they are IDOR-safe; an unknown id is a silent no-op that still returns 200).

Method + path	Purpose	Main tables
GET `/candidates/me`	Profile summary (identity, CTC, notice, skills, education, experience, resume meta)	`tblSubscriberCVDetails`, `tblSubscriberProfileExtra`, education, employer, tags, CV uploaded
POST `/candidates/me/complete-onboarding`	Sets `onboardedAt` once; returns `{isOnboarded:true}`	`tblSubscriberCVDetails`
GET `/candidates/me/cv-masters`	Lookup lists for the editor	master tables
GET `/candidates/me/institutes?q&stateId&limit`	Institute typeahead (parameterised raw SQL)	`tblMstrInstitute`
GET `/candidates/me/cv-edit`	Full editable profile (11 tables)	subscriber tables
PUT `/candidates/me/personal`	Name, email (no re-verification), mobile, DOB, gender, address, city. First save also writes `CV_CREATED` history	CV details, registration, status history
PUT `/candidates/me/professional`	Function, experience, CTC, notice, industry, preferred cities, tags	CV details, profile extra, preferred locations, tags
PUT / DELETE `/candidates/me/education[/:id]`	Education rows (degree/course consistency, year ranges, marks)	`tblSubscriberEducation`
PUT / DELETE `/candidates/me/employment[/:id]`	Employment history	`tblSubscriberEmployer`
PUT / DELETE `/candidates/me/certificates[/:id]`	Certificates	`tblSubscriberCertificate`
PUT `/candidates/me/headline` · `/summary` · `/key-skills` · `/career-profile` · `/personal-details` · `/diversity`	Profile sections	`tblSubscriberProfileExtra` (+ tags for key skills)
PUT / DELETE `/candidates/me/it-skills` · `/projects` · `/accomplishments` · `/languages[/:id]`	Child sections	matching subscriber tables
GET `/candidates/me/applied-jobs`	The candidate's applications with status history and latest interview	`tblJobSubscriberMapping` + status tables
GET `/candidates/me/documents`	Requested / uploaded documents	`tblCandidateDocumentMap`, `tblCandidateDocumentUploaded`
POST `/candidates/me/documents/:documentTypeId`	Upload a requested document (multipart `file`) — 400 if not requested	storage + `tblCandidateDocumentUploaded`
GET / POST `/candidates/me/job-alerts`	Stored job alerts (no update/delete; **no sender**)	`tblSubscriberJobAlert`
GET `/candidates/me/saved-jobs`, `/saved-job-ids`; POST / DELETE `/candidates/me/saved-jobs/:jobId`	Saved jobs	`tblSavedJob`
POST `/candidates/me/change-password`	Verify current, set new (min 8). Does not revoke other sessions	`tblSecUser`
GET `/candidates/me/dashboard`	`{appliedCount, savedCount, interviewCount, profileCompletion}`	counts
GET / PUT `/candidates/me/notification-prefs`	Preferences. **PUT persists only 4 of 12 fields**	`tblNotificationPreference`
GET / POST / DELETE `/candidates/me/saved-searches[/:id]`	Saved searches	`tblSavedSearch`
GET `/candidates/me/recommendations`	Up to 20 active jobs matching skill tag, city or industry	jobs
GET `/candidates/me/activity?cursor`	Merged activity timeline (page 20)	5 sources
GET / POST `/candidates/me/referrals`	Refer-a-friend records (no invite is sent)	`tblCandidateReferral`
GET / PUT `/candidates/me/privacy`	Privacy flags (stored, not enforced elsewhere)	`tblCandidatePrivacy`
POST `/candidates/me/export` (202)	Records an export request (no processor)	`tblCandidatePrivacy`
POST `/candidates/me/delete-account` (202)	Records deletion request, deactivates the login, deletes sessions (transaction)	privacy, `tblSecUser`, sessions

4. Files — `/api/files`

Controller `apps/api/src/modules/files/files.controller.ts`; storage in `apps/api/src/modules/storage/*`.

Method + path	Auth	Purpose
POST `/files/upload/:documentTypeId`	**Any**	Generic upload into a document-type folder; not linked to any record; unused by the web app

Method + path	Auth	Purpose
GET `/files/:folder/:sub/:name`	**Any (no ownership check)**	Download any stored file by key (local driver). See TESTING_AND_ISSUES §1
POST `/files/resume`	Candidate, 10/min	Upload own resume (also records size and PDF page count)
GET `/files/resume`	Candidate	Download own resume
DELETE `/files/resume`	Candidate	Clears DB references (stored object is not deleted)
POST `/files/avatar`	Candidate, 10/min	Upload profile photo
GET `/files/avatar/:subscriberId`	Public, 300/min	Serve a candidate photo (jpg/png only), cached 1h

Upload rules (`storage.service.ts`): ≤10 MB (checked after buffering), MIME in pdf/jpeg/png/doc/docx and extension must match; key = `<RootFolder>/<FolderName>/<userId>-<timestamp>-<random>.<ext>` . Driver: `STORAGE_DRIVER=local` (disk, served by the API) or `s3` (private bucket, 300 s presigned URLs).

---

5. Jobs — `/api/jobs`

Controller `apps/api/src/modules/jobs/jobs.controller.ts` ; services `jobs.service.ts` , `job-application.service.ts` .

Method + path	Auth	Purpose / notes
GET `/jobs/filters`	Public	Filter option lists (5-min in-process cache)
GET `/jobs`	Public	Structured search over **Active** jobs: designation, industry, location, workMode, employmentType, min/max exp & CTC, comma lists (`workModes` , `employmentTypes` , `locations` , `states` , `skills`), `postedWithin` (24h/7d/30d), `sortBy` , `page` , `pageSize` (default 10). Returns `{rows, total}`
GET `/jobs/search`	Public	Postgres full-text search (`to_tsvector` / `to_tsquery` , ts_rank) over Active jobs; empty `q` falls back to `/jobs`
GET `/jobs/suggestions?q`	Public	Up to 5 typeahead suggestions (designation / company / skill)
GET `/jobs/recommended`	Candidate	Delegates to candidate recommendations
GET `/jobs/:id`	Public	Job detail. **Not filtered by status** (Draft/Closed/Archived are readable)
POST `/jobs/:id/apply`	Candidate	Apply. Requires an **active paid subscription** (403 otherwise), job Active (404), not already applied (400). Body: optional application-form snapshot (name, email, phone, experience, location, expected salary, notice, LinkedIn, portfolio, resume file name, cover letter). Returns `{jobSubscriberMapId, reference:'AJ-000123'}`

Apply transaction (`JobApplicationsService.apply`): creates `tblJobSubscriberMapping` (status 1 MAPPED), a `tblJobSubscriberStatus` log row, upserts `tblSubscriberJobStatusLatest` , writes `tblSubscriberStatusHistory` (5 MAPPED_TO_JOB) and optionally `tblJobApplicationDetail` ; then audit `job.applied` . No email is sent.

---

6. Employers — `/api/clients`

Controller `apps/api/src/modules/employers/employers.controller.ts` , class-level `@Roles(Client, Admin)` ; service `employers.service.ts` . (`apps/api/src/modules/clients/*` only re-exports these and is not imported by `AppModule` .)

Ownership: every `me/*` call resolves the company via `tblSecUser.NodeID → tblMstrPerson.ClientID` (`clientIdFor`) and returns 404 when missing. Jobs are checked with `ownedJob` ; applications by `mapping.job.clientID` . Admin is listed in `@Roles` but has no linked company, so **Admin receives 404 on every `me/*` route**.

Method + path	Purpose	Notes
GET / PATCH `/clients/me`	Company profile (+ HR contact)	PATCH accepts any string as `companyLogo` (see issues)
POST `/clients/me/logo`	Upload logo (JPEG/PNG)	
GET `/clients/masters`	Lookups for job forms	
GET `/clients/me/jobs`	List own jobs with counts by status and filters `q, city, status, page, pageSize`	Draft/Archived hidden unless filtered

Method + path	Purpose	Notes
POST `/clients/me/jobs?draft=`	Create job (Active, or Draft with `draft=true`)	`$transaction` (job + skills); no audit; no min≤max checks
GET / PATCH / DELETE `/clients/me/jobs/:id`	Read / edit / delete (delete only if no applications)	
POST `/clients/me/jobs/:id/status`	`{status: Active	Closed
POST `/clients/me/jobs/:id/deactivate` · `/archive` · `/activate`	Shortcuts to the above	
POST `/clients/me/jobs/:jobId/duplicate`	Copy as a new **Active** job	Drops two text fields
POST `/clients/me/jobs/bulk-upload?commit=`	CSV import: validate preview, then commit	Per-row transactions
GET `/clients/me/applicants`	All applications on own jobs (filters status, q, jobId, city, minExp, maxNotice)	In-memory pagination; includes pre-screening applicants with contact details
GET `/clients/me/applicants/:id`	Full applicant profile and timeline	
GET `/clients/me/applicants/:id/resume?inline=`	Applicant CV	
POST `/clients/me/applicants/:id/decision`	`{decision: Shortlisted	Hired
POST `/clients/me/applicants/:id/decision/undo`	Revert Shortlisted/Rejected	
GET / POST `/clients/me/applicants/:id/interview-rounds`	List / request first round (exactly 3 distinct future slots, interviewer + HR details, mode)	Q3 then confirms a slot
POST `/clients/me/applicants/:id/interview-rounds/:roundId/result`	`{decision: Select	Reject
GET / PUT `/clients/me/applicants/:id/notes`	Private notes (PUT appends)	
GET `/clients/me/applicants/:id/documents`	Candidate documents	Scoped to the candidate, not the application
POST `/clients/me/applicants/:id/documents/review`	`{docUploadId, status: Approved	NeedsCorrection, comments?}`
GET `/clients/me/document-types`	Requestable document types	
POST `/clients/me/applicants/:id/document-request`	Request documents (only when status = SELECTED)	
GET `/clients/me/analytics` · `/analytics/export`	Funnel / job performance, CSV export	
GET `/clients/me/billing` · `/billing/export`	Invoices (read-only), CSV export	
GET / PATCH `/clients/me/branding`	Branding (unused by web)	
POST `/clients/me/interviews/generate-link`	Returns a placeholder `meet.aajiveka.com` URL	Not a real meeting
GET `/clients/:id/logo`	**Public** company logo	Streams the stored key
GET `/clients/:id/public`	**Public** company card	

---

7. Recruitment — `/api/recruitment`

Controller `apps/api/src/modules/recruitment/recruitment.controller.ts`, class-level `@Roles(QC1, QC2, Admin)`; method-level `@Roles` override it. Services: `recruitment.service.ts`, `cv-referral.service.ts`, `interview-round.service.ts`, `offer-letter.service.ts`, `scoring.service.ts`, `mapping-access.service.ts` (ownership checks for Candidate and Client only).

Method + path	Roles	Purpose
GET `/recruitment/candidates?search&status&page&pageSize`	QC1, QC2, Admin	Candidate list
GET `/recruitment/candidates/:id`	QC1, QC2, Admin	Candidate detail + registration status + latest score
GET `/recruitment/candidates/:id/resume`	QC1, QC2, Admin	Candidate CV
POST `/recruitment/candidates/:id/decision`	QC1, Admin	Registration decision: Approved / Rejected / OnHold / NeedMoreInfo / Duplicate / Withdrawn (does **not** change any application status)
POST `/recruitment/candidates/:id/assign-job`	QC1, QC2, Admin	Staff-created application (same as apply, without the subscription gate)

Method + path	Roles	Purpose
POST `/recruitment/candidates/:id/documents`	QC2, Admin	Assign document types to collect
GET `/recruitment/qc1/stats`	QC1, Admin	Legacy QC1 counters
GET `/recruitment/jobs`	QC1, QC2, Admin	Active jobs
GET `/recruitment/document-types`	QC1, QC2, Admin	Candidate-uploadable document types
GET `/recruitment/interviews`, `/interviews/eligible`	QC1, QC2, Admin	Legacy interview list (tblJobInterviewStatus)
POST `/recruitment/interviews`	QC1, QC2, Admin	Legacy schedule → status 3, email
POST `/recruitment/interviews/:id/status`	QC1, QC2, Admin	Completed → 6, Cancelled → 9
GET `/recruitment/interview-modes`	QC1, QC2, Q3, Client, Admin	Lookup
GET `/recruitment/documents` · `/documents/:documentId/file`	QC1, QC2, Admin	All candidate documents (unscoped) / download
POST `/recruitment/documents/review`	QC1, QC2, Admin	Verified / Rejected
PATCH `/recruitment/pipeline/:id`	QC1, QC2, Admin	Sets any status with no log/history/audit. **Unused by the web app**
POST `/recruitment/applications/:id/score`	QC1, Admin	Compute and store the 7-factor match score
POST `/recruitment/referrals`	QC2, Admin	Refer applications to Q3 (→ 14)
GET `/recruitment/referrals?status`	QC2, Q3, Admin	All referrals
POST `/recruitment/referrals/forward`	Q3, Admin	Forward to company (→ 15, 14-day window)
POST `/recruitment/referrals/:id/company-review`	Client, Admin	Company shortlist (creates round 1 with 3–4 slots) or reject
POST `/recruitment/interview-rounds`	Q3, Client, Admin	Create a round (any roundNumber)
GET `/recruitment/interview-rounds/:mapId`	QC2, Q3, Client, Candidate, Admin	Rounds + slots (Client/Candidate ownership-checked)
POST `/recruitment/interview-rounds/:roundId/slots`	Q3, Client, Admin	Replace unselected slots (resets round to Pending)
POST `/recruitment/interview-rounds/:roundId/select-slot`	Candidate, Q3, Admin	Pick a slot → round Scheduled, emails
POST `/recruitment/interview-rounds/:roundId/result`	Q3, Client, Admin	Passed / Failed / Hold (+ next round or Select)
POST `/recruitment/offers`	Q3, Client, Admin	Create offer (needs SELECTED and all docs verified)
POST `/recruitment/offers/:id/send`	Q3, Client, Admin	Send offer → 20, email
POST `/recruitment/offers/:id/respond`	Candidate, Admin	Accept (→ 21) / reject
POST `/recruitment/offers/:id/mark-joined`	Q3, Client, Admin	→ 22 JOINED
GET `/recruitment/offers/:mapId`	QC2, Q3, Client, Candidate, Admin	Offer for an application

8. Q1 screening — `/api/q1`

Controller `apps/api/src/modules/q1/q1.controller.ts`, `@Roles(QC1, Admin)`. Services `q1.service.ts`, `q1-analytics.service.ts`. Screening state lives in `tblSubscriberScreening` (New, Screening, Incomplete, FollowUp, NoResponse, Verified, NotInterested).

Method + path	Purpose
GET `/q1/stats`, `/q1/nav-counts`	Dashboard counters
GET `/q1/candidates?tab&search&priority&experience&cvOnly&page&pageSize`	Screening queue (tabs: all, new, screening, incomplete, follow-up, no-response, verified, not-interested)
GET `/q1/candidates/:id`	Candidate profile + completeness + screening row (opening moves New → Screening)
GET `/q1/candidates/:id/contact-log`	Last 50 contact entries
PATCH `/q1/candidates/:id/checklist` · `/checklist/all`	Toggle 7 checklist items

Method + path	Purpose
POST `/q1/candidates/:id/verify`	Mark Verified (no precondition)
POST `/q1/candidates/:id/contact` · `/request-update` · `/request-cv`	Log a contact attempt (Call/WhatsApp/Email/Message) — **nothing is actually sent**
POST `/q1/candidates/:id/status`	FollowUp (date/time) / NoResponse (attempts) / NotInterested (reason)
GET `/q1/analytics`	Screening analytics

---

9. Q2 matching — `/api/q2`

Controller `apps/api/src/modules/q2/q2.controller.ts`, `@Roles(QC2, Admin)`. Services `q2.service.ts`, `q2-analytics.service.ts`. Review state in `tblApplicationMatchReview` (New, Forwarded, SentBackToQ1, Rejected).

Method + path	Purpose
GET `/q2/stats`, `/q2/nav-counts`, `/q2/sla`	Counters (the "SLA" is a backlog count, not time-bound)
GET `/q2/jobs?search`	Active jobs with applicant/relevant counts (lazily scores up to 25 applications per call)
GET `/q2/jobs/:jobId?relevantOnly`	Ranked applicants for a job
GET `/q2/applicants?tab&bucket&search&page&pageSize`	All applicants (tabs all/relevant/notRelevant; buckets queue/forwarded/sentBack/rejected)
GET `/q2/applications/:mapId`	Application detail with 7-criterion score breakdown and skill coverage (creates review row, stamps first view)
PATCH `/q2/applications/:mapId/criteria`	Include/exclude a criterion from the displayed total
POST `/q2/applications/:mapId/forward`	Refer to Q3 (→ 14)
POST `/q2/applications/:mapId/send-back`	Send back to Q1 (→ 11 NEED_MORE_INFO, screening → Incomplete)
POST `/q2/applications/:mapId/reject`	Reject (→ 5)
GET `/q2/analytics`	Matching analytics

Score weights (`apps/api/src/shared/matching.ts`, duplicated in `scoring.service.ts`): skills 30%, experience 25%, job role 20%, education 10%, location / salary / notice 5% each. Relevant = score ≥ 60.

---

10. Q3 hiring operations — `/api/q3`

Controller `apps/api/src/modules/q3/q3.controller.ts`, `@Roles(Q3, Admin)`. Services `q3-read.service.ts`, `q3-referrals.service.ts`, `q3-interviews.service.ts`, `q3-documents.service.ts`, `q3-mail.ts`. Only applications with at least one `tblCvReferral` row are visible.

Method + path	Purpose
GET `/q3/nav-counts`, `/q3/dashboard`, `/q3/pipeline?stage&search&company`, `/q3/report?quarter`, `/q3/queues/:name`	Read models (stage derived by `stageOf` in `shared/hiring.ts`)
GET `/q3/referrals`	Referred CVs grouped by job batch
POST `/q3/referrals/forward`	Forward referrals to the company (all must have a CV)
POST `/q3/referrals/return`	Return to Q2 with reason (+ optional email to Q2)
GET `/q3/company-review?tab`	14-day company review board
POST `/q3/company-review/remind-expiring`	Email companies with CVs about to expire
GET `/q3/applications/:mapId/shortlist`	Shortlist summary
PUT `/q3/applications/:mapId/round-setup`	Capture company shortlist + round panel + slots (→ 2 SHORTLISTED) or edit a pending round
GET `/q3/applications/:mapId/scheduling`	Scheduling workspace
POST `/q3/applications/:mapId/contact`	Log a candidate contact
POST `/q3/rounds/:roundId/slots/:slotId/availability?unavailable`	Mark a slot (un)available
POST `/q3/rounds/:roundId/request-slots`	Email the company for slots
POST `/q3/meeting-link`	Generate a Jitsi link
POST `/q3/rounds/:roundId/confirm`	Confirm a slot → round Scheduled, status R1..R3/Final, invitation emails

Method + path	Purpose
GET `/q3/interviews?week`	Weekly interview board
POST `/q3/rounds/:roundId/remind` · `/checklist` · `/reschedule` · `/conducted`	Interview-day actions
GET `/q3/applications/:mapId/rounds`	Rounds and decisions
POST `/q3/applications/:mapId/nudge`	Chase the company for a result
POST `/q3/rounds/:roundId/decision`	NextRound / FinalRound / Hold / Rejected / Selected (Selected only on final round; creates document requests)
POST `/q3/rounds/:roundId/decision-mail`	Send the decision email
GET `/q3/applications/:mapId/documents`	Document checklist + offer
POST `/q3/documents/:docUploadId/verify` · `/send-back`	Verify or send back a document
GET `/q3/documents/:docUploadId/file?preview`	View/download a document
POST `/q3/applications/:mapId/documents/request` · `/forward` · `/approve`	Request docs (due in 3 days) → forward to company → approve
POST `/q3/applications/:mapId/offer`	Generate and send offer (needs docs approved) → 20
POST `/q3/applications/:mapId/joined`	Mark joined (offer must be Accepted) → 22
GET `/q3/returned?tab`, `/q3/returned/:referralId/mail`; POST `/q3/returned/:referralId/resend`	Returned-to-Q2 tracker

11. Admin — `/api/admin`

Controller `apps/api/src/modules/admin/admin.controller.ts`, `@Roles(Admin)`. Service `admin.service.ts`, `staff-account.ts`. The admin panel is **read-only over the hiring pipeline** by design; pipeline actions stay in the Q1/Q2/Q3/employer workspaces (which also admit Admin).

Method + path	Mutates	Purpose
GET `/admin/stats`	—	Platform KPIs, signups (30 d), top designations/cities, recent audit
GET `/admin/users?page&pageSize&q&roleId&isActive`	—	User search
POST `/admin/users`	yes	Create staff login (QC1, QC2, Q3, Admin) + emailed set-password link
GET `/admin/users/:userId`	—	User detail (logins, sessions, audit, applications, company)
POST `/admin/users/:userId/password-reset`	yes	Email a 1-hour reset link
PATCH `/admin/users/:userId`	yes	Active flag and/or role (never self; never across candidate/employer/staff lines); revokes refresh sessions
POST `/admin/users/bulk-update` · `/bulk-delete`	yes	Bulk active/role; bulk soft-delete (bulk-delete unused by web)
GET `/admin/jobs`, `/admin/jobs/:jobId`	—	Jobs and job detail
POST `/admin/jobs/:jobId/status` · `/admin/jobs/bulk-moderate`	yes	`close` / `reopen` (Active ↔ Closed only)
GET `/admin/applications`, `/admin/applications/:mapId`	—	Applications by status/stage with timeline and next-step deep link
GET `/admin/companies`	—	Companies with job/application counts
GET `/admin/audit-logs?action&entity&entityId&userId`	—	Audit log browser
GET `/admin/employer-registrations?status`	—	Registration queue
POST `/admin/employer-registrations/:id/review`	yes	Approve (creates company + login + emails a temporary password) or reject
GET / PATCH `/admin/settings`	yes	18 platform settings — **stored but read by no other code**
GET / POST `/admin/blog-posts`; PATCH / DELETE `/admin/blog-posts/:id`	yes	Blog CMS

12. Payments — `/api/payments`

Controller `apps/api/src/modules/payments/payments.controller.ts`; services `payments.service.ts`, `razorpay.service.ts`, `billdesk.service.ts`.

Method + path	Auth	Purpose
GET `/payments/plans`	Public	Active subscription plans (12 seeded: 3 tiers × 1/3/6/12 months)

Method + path	Auth	Purpose
POST `/payments/create-order`	Candidate, 10/min	Create a Razorpay order `{planId}` → `{orderRef, razorpayOrderId, amount, currency, keyId, planName}`
POST `/payments/verify`	Candidate, 10/min	Verify Razorpay checkout signature → order SUCCESS + `tblSubscription` (transaction, idempotent)
POST `/payments/webhook`	Public (HMAC)	Razorpay webhook (`x-razorpay-signature`); handles `payment.captured` / `payment.failed`
GET `/payments/history`	Candidate	Own orders
POST `/payments/orders`	Candidate, 10/min	Create a **BillDesk** order (legacy path) → `{orderRef, amountInr, redirectUrl}`
GET `/payments/orders/:orderRef`	Candidate	Order status (used by `/payment/return`)
GET `/payments/subscription`	Candidate	Current subscription

Effect of payment: only a `tblSubscription` row (start = end of current subscription or now; stacks). It gates **candidate self-apply** only; the user's role is not changed. **`PaymentsService.settle()` (BillDesk settlement) has no route**, so BillDesk orders can never complete. No receipt email is sent.

---

13. Exports — `/api/exports`

Controller `apps/api/src/modules/exports/exports.controller.ts` . Query `format=csv|xlsx|pdf` (default csv). Every export writes an `export.<name>` audit row.

Method + path	Roles	Data	Known defects
GET `/exports/candidates?search&status`	QC1, QC2, Admin	Up to 5,000 candidates	not used by web
GET `/exports/applicants`	Client, Admin	Employer applicants	**only 10 rows** (default page size)
GET `/exports/users?roleId&isActive&from&to`	Admin	Users	date filter = users with a login in range
GET `/exports/jobs?status&company&from&to`	Admin	Jobs	`status` , `company` ignored
GET `/exports/applications?jobId&from&to`	Admin	Applications with contact details	
GET `/exports/payments?from&to`	Admin	Payment orders	

---

14. Notifications — `/api/notifications`

Controller `apps/api/src/modules/notifications/notifications.controller.ts` (any authenticated role).

Method + path	Purpose
GET `/notifications/stream`	Server-Sent Events stream (`connected` event + 30 s heartbeat). Web client uses `fetch` streaming with the bearer header
GET `/notifications?page&pageSize&type&unread`	In-app notifications
PATCH `/notifications/:id/read` , `/notifications/read-all`	Mark read
DELETE `/notifications/:id`	Delete

The store is **in-memory and nothing in the codebase creates notifications**, so these endpoints always return empty data.

---

15. Companies, site documents, health

Method + path	Auth	Purpose	File
GET `/companies/search?q`	Public	Company typeahead (20)	`modules/companies/companies.controller.ts`
GET `/companies/:id`	Public	Company page incl. contact details and active job count	same
GET `/site-docs`	Public	Published downloadable documents	`modules/site-documents/site-documents.controller.ts`

Method + path	Auth	Purpose	File
GET `/site-docs/:slug`, `/site-docs/:slug/download`	Public	Serve a document (inline for pdf/images; S3 → 302 presigned)	same
GET `/health`	Public, unthrottled	`{status ok	degraded, env, db, redis, disk, timestamp}` — always HTTP 200

Site documents are written only by the CLI `apps/api/scripts/publish-site-document.ts`.

---

16. Background jobs (BullMQ)

Queue	Job	Schedule	Processor	Effect
`notifications`	`email`, `sms`	on demand (5 attempts, exponential backoff from 2 s)	`apps/api/src/modules/notifications/notifications.worker.ts` (in-process)	Delivers through the configured provider
`cv-expiry`	`expire-stale-cvs`	every hour (scheduler upserted in `RecruitmentModule.onModuleInit`)	`apps/api/src/modules/recruitment/cv-expiry.processor.ts` → `CvReferralService.expireStale`	Expires CVs not reviewed by the company within 14 days (→ 23 CV_EXPIRED), puts them back in Q2's queue, emails Q2

There are no other cron jobs, webhooks consumers or retention processors.

---

17. External integrations

Integration	Env vars (names only)	Status
PostgreSQL 16 (Prisma 7, `@prisma/adapter-pg`)	`DATABASE_URL`	Required
Redis (OTP, BullMQ)	`REDIS_URL`	Required
Email: SMTP (nodemailer) / AWS SES / log	`EMAIL_DRIVER`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_SECURE`, `SMTP_FROM`, `SMTP_FROM_NAME`, `SES_FROM_EMAIL`, `AWS_REGION`	Default `log` (emails are logged, not sent)
SMS: 2Factor / MSG91 / AWS SNS / log	`SMS_DRIVER`, `TWOFACTOR_API_KEY`, `MSG91_AUTH_KEY`, `MSG91_TEMPLATE_ID`, `SNS_SENDER_ID`, `SNS_DLT_ENTITY_ID`, `SNS_DLT_TEMPLATE_ID`	Providers implemented, **but no code sends SMS**
Storage: local disk / AWS S3	`STORAGE_DRIVER`, `STORAGE_LOCAL_ROOT`, `S3_BUCKET`, `AWS_REGION`	Implemented
Razorpay	`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`	Wired to the UI
BillDesk	`BILLDESK_BASE_URL`, `BILLDESK_CLIENT_ID`, `BILLDESK_MERCHANT_ID`, `BILLDESK_SECRET_KEY`	Order creation only; settlement unreachable; never run against the real gateway
Jitsi	—	Q3 generates `meet.jit.si` links (no API)
Google Analytics / web vitals	web-side (`apps/web/src/lib/ga4.ts`, `webVitals.ts`)	Client-side only

---

18. Frontend calls with no backend route

Verified by grepping `apps/web/src` against the controllers:

Frontend call	Caller	Result
GET `/candidates/:id/profile`	`apps/web/src/features/candidates/pages/PublicProfilePage.tsx`	404
GET / POST `/companies/:id/reviews`	`apps/web/src/features/public/pages/CompanyReviewsPage.tsx`	404
GET / POST `/clients/:id/reviews`	`apps/web/src/features/public/components/CompanyReviews.tsx` (rendered on `/companies/:id`)	404
GET `/jobs/salary-insights`	`apps/web/src/features/public/pages/SalaryInsightsPage.tsx`	400 (captured by `/jobs/:id` → ParseIntPipe)
POST `/clients/me/campaigns/bulk`	`apps/web/src/features/clients/components/BulkEmailUpload.tsx` (orphan component, not routed)	404

See FRONTEND_BACKEND_MAPPING.md for the page-by-page mapping.